CAPÍTULO 9. GIT Y OTROS SISTEMAS

Introducción

Git es un sistema de control de versiones distribuido que permite registrar los cambios realizados en los archivos de un proyecto. Aunque muchas personas utilizan Git para administrar sus proyectos directamente, en el entorno profesional también existen proyectos que utilizan otros sistemas de control de versiones, como Subversion, Mercurial, Perforce o Team Foundation Version Control (TFVC).

El capítulo 9 de *Pro Git* explica cómo utilizar Git para interactuar con proyectos que se administran mediante otros sistemas y cómo trasladar proyectos existentes a Git. De esta manera, los equipos pueden aprovechar las características de Git sin tener que abandonar inmediatamente todas las herramientas que utilizan.

El capítulo se divide en dos temas principales: el uso de Git como cliente de otros sistemas de control de versiones y la migración de proyectos a Git.

1. Git como cliente de otros sistemas

En algunos proyectos, el equipo de trabajo utiliza un sistema de control de versiones diferente de Git. En estos casos, es posible utilizar herramientas de integración conocidas como *puentes*, que permiten que Git se comunique con otros sistemas.

Estas herramientas facilitan el trabajo con repositorios externos, aunque su funcionamiento depende del sistema de origen y de las características de cada herramienta. No todos los sistemas representan las ramas, las revisiones y los metadatos de la misma manera, por lo que pueden existir diferencias al intercambiar información.

- Git y Subversion

Subversion, también conocido como SVN, es un sistema de control de versiones centralizado. A diferencia de Git, en el que cada clon contiene el historial del repositorio, Subversion utiliza principalmente un servidor central que almacena las revisiones del proyecto.

Git puede interactuar con repositorios Subversion mediante la herramienta git svn. Esto permite trabajar localmente con Git y comunicarse con el servidor SVN cuando sea necesario.

Entre los comandos principales se encuentran:

- git svn clone: permite crear un repositorio Git a partir de un repositorio Subversion.
- git svn fetch: obtiene nuevas revisiones del repositorio Subversion.
- git svn rebase: incorpora las revisiones nuevas del servidor a la rama local mediante una reorganización del trabajo local.

- git svn dcommit: convierte las confirmaciones locales compatibles en nuevas revisiones de Subversion.

Este mecanismo puede ser útil cuando una organización todavía trabaja con SVN, pero algunos integrantes prefieren utilizar las herramientas locales de Git.

Sin embargo, la integración requiere comprender las diferencias entre ambos sistemas. Por ejemplo, una confirmación de Git no se transforma necesariamente en una confirmación equivalente de SVN hasta que se envía al servidor mediante el procedimiento correspondiente.

- **Git y Mercurial**

Mercurial es otro sistema de control de versiones distribuido. Comparte algunas características con Git, como el trabajo local y el registro de versiones, pero utiliza sus propios comandos y estructuras internas.

Para interactuar con repositorios Mercurial desde Git existen herramientas de integración, como git-remote-hg. Estas herramientas permiten establecer una comunicación entre ambos sistemas y facilitar el intercambio de revisiones.

El procedimiento general consiste en instalar y configurar la herramienta de integración, acceder al repositorio Mercurial y utilizar Git para trabajar con el contenido y las revisiones que la herramienta pueda representar.

La compatibilidad depende de la herramienta y de su versión. Por ello, antes de utilizar este método en un proyecto importante, es recomendable comprobar que la solución elegida sigue siendo compatible con el entorno de trabajo.

- **Git y Perforce**

Perforce es un sistema de control de versiones utilizado en proyectos de desarrollo de software y otros entornos en los que se administran grandes cantidades de archivos.

Git puede interactuar con Perforce mediante herramientas como git-p4 y Perforce Git Fusion.

git-p4 permite intercambiar información entre un repositorio Git y un servidor Perforce. Dependiendo de la configuración y de las opciones utilizadas, puede importar contenido y revisiones de Perforce a Git, así como facilitar el envío de cambios desde Git hacia Perforce.

Por su parte, Perforce Git Fusion ofrece mecanismos de integración para trabajar con repositorios Git y datos administrados por Perforce.

Estas herramientas pueden resultar útiles cuando un equipo desea utilizar Git localmente, pero necesita mantener la compatibilidad con una infraestructura que todavía depende de Perforce.

- **Consideraciones al trabajar con sistemas externos**

El uso de Git como cliente de otro sistema requiere tener en cuenta varios aspectos:

1. **Compatibilidad:** la herramienta de integración debe funcionar con el sistema de origen y con la versión instalada.
2. **Historial:** es necesario comprobar cómo se representan las revisiones y las confirmaciones entre los sistemas.
3. **Autores:** los nombres de usuario y los datos de los responsables pueden requerir ajustes para mantener una identificación consistente.
4. **Ramas:** la forma de representar y sincronizar las ramas puede variar entre sistemas.
5. **Sincronización:** el equipo debe establecer un procedimiento claro para obtener cambios y enviar modificaciones.
6. **Pruebas:** antes de utilizar la integración en un proyecto real, conviene comprobarla con una copia de prueba.

Estas precauciones ayudan a evitar pérdidas de información y dificultades de coordinación entre los integrantes del equipo.

2. Migración de proyectos a Git

La migración consiste en trasladar un proyecto administrado mediante otro sistema de control de versiones a un repositorio Git.

Este proceso puede realizarse cuando una organización decide adoptar Git como herramienta principal. La migración no consiste únicamente en copiar los archivos: también puede ser necesario conservar el historial, las ramas, las etiquetas y los datos de los autores.

La estrategia apropiada depende del sistema de origen, del tamaño del proyecto y de la cantidad de información histórica que se desea conservar.

- **Migración desde Subversion**

Para trasladar un proyecto de Subversion a Git se puede utilizar git svn u otras herramientas de importación.

Un aspecto importante consiste en conservar correctamente la información de los autores. En SVN, los responsables de las revisiones pueden estar identificados mediante nombres de usuario, mientras que en Git las confirmaciones suelen incluir un nombre y una dirección de correo electrónico.

Por esta razón, puede ser necesario crear un archivo de correspondencias que relacione los usuarios del sistema anterior con los datos que se utilizarán en Git.

También es importante comprobar que las revisiones, las ramas y las etiquetas relevantes se hayan importado correctamente.

Una vez finalizada la migración, se debe verificar el historial y decidir cómo se continuará trabajando. Si la organización cambia definitivamente a Git, puede establecer un nuevo repositorio remoto para que el equipo empiece a publicar sus cambios allí.

- **Migración desde Mercurial**

La migración desde Mercurial puede realizarse mediante herramientas especializadas, como hg-fast-export, que permiten convertir información del repositorio de origen en objetos y confirmaciones de Git.

El procedimiento general incluye:

1. Crear una copia del repositorio Mercurial.
2. Identificar a los autores y preparar sus equivalencias, si es necesario.
3. Ejecutar la herramienta de conversión.
4. Revisar las confirmaciones, las ramas y las etiquetas importadas.
5. Verificar que los archivos y el historial sean correctos.
6. Configurar el repositorio remoto de Git y publicar las ramas que deban compartirse.

La conversión debe revisarse cuidadosamente, ya que los dos sistemas no siempre representan de manera idéntica todos los elementos del historial.

- **Migración desde Perforce**

Para importar un proyecto de Perforce a Git pueden utilizarse herramientas como git-p4 o Perforce Git Fusion.

Estas herramientas permiten trasladar contenido y, según la configuración, parte del historial del proyecto. En la planificación de la migración es importante determinar qué ramas se deben importar, cómo se identificarán los autores y qué información debe conservarse.

Después de la importación, es necesario comprobar que los archivos, las revisiones y las ramas importantes estén disponibles en el nuevo repositorio.

También debe definirse el momento en que el equipo dejará de realizar modificaciones en el sistema anterior, si la intención es adoptar Git como sistema principal. Una transición organizada reduce el riesgo de que se generen cambios separados en dos repositorios diferentes.

- **Migración desde Team Foundation Version Control**

Team Foundation Version Control, conocido como TFVC, es un sistema de control de versiones centralizado asociado a herramientas de Microsoft.

La migración desde TFVC a Git puede realizarse mediante herramientas como git-tfs, dependiendo del entorno y de las versiones utilizadas.

Uno de los aspectos importantes de este procedimiento es la correspondencia de los usuarios. La herramienta debe poder relacionar los responsables de los cambios del sistema anterior con los nombres y las direcciones de correo electrónico que se utilizarán en Git.

También deben revisarse las ramas y las confirmaciones importadas, ya que la estructura del sistema de origen puede diferir de la que utiliza Git.

Antes de ejecutar una migración definitiva, es recomendable realizar pruebas, comparar los resultados con el repositorio original y conservar una copia de seguridad.

- **Migración desde otros sistemas**

No todos los proyectos utilizan los sistemas mencionados anteriormente. Algunas organizaciones pueden trabajar con CVS, ClearCase, Visual SourceSafe u otras herramientas de control de versiones.

En estos casos, es conveniente buscar una herramienta de importación compatible con el sistema de origen. Si no existe una solución adecuada, Git ofrece una herramienta avanzada llamada git fast-import.

Esta herramienta permite importar información mediante instrucciones que describen los objetos, las confirmaciones, las ramas y los archivos que deben crearse en el repositorio Git.

Para utilizarla, se puede desarrollar un programa que lea la información del sistema de origen y genere las instrucciones requeridas. Este método ofrece flexibilidad, pero exige conocimientos técnicos y pruebas cuidadosas para evitar errores en la reconstrucción del historial.

La migración personalizada resulta especialmente útil cuando el sistema original no cuenta con un importador adecuado o cuando es necesario adaptar la información a una estructura específica.

3. **Importancia de conservar el historial**

Uno de los objetivos principales de una migración es conservar la información que permite comprender la evolución del proyecto.

El historial puede incluir los cambios realizados, los responsables, las fechas, las ramas y las etiquetas. Estos elementos ayudan a investigar errores, identificar versiones anteriores y comprender por qué se tomaron determinadas decisiones.

Sin embargo, la conservación completa depende de las capacidades de la herramienta de importación y de la estructura del sistema original. Por eso, no se debe asumir que una migración ha sido exitosa únicamente porque los archivos actuales aparecen en Git.

Es necesario verificar el historial y comparar los resultados con el repositorio de origen antes de dar por terminado el proceso.

4. Buenas prácticas durante la migración

Para realizar una migración organizada a Git, se recomienda:

- Crear una copia de seguridad del repositorio original.
- Identificar las ramas, etiquetas, autores y revisiones que deben conservarse.
- Elegir una herramienta compatible con el sistema de origen.
- Realizar una migración de prueba antes de modificar el flujo de trabajo del equipo.
- Revisar los archivos y el historial del repositorio resultante.
- Documentar los problemas encontrados y las soluciones aplicadas.
- Establecer una fecha o procedimiento claro para comenzar a trabajar en Git.
- Evitar que se realicen cambios simultáneos en sistemas diferentes sin un procedimiento de sincronización.

Estas prácticas permiten reducir los riesgos y facilitan que los integrantes del equipo se adapten al nuevo sistema.

Ejercicio práctico

Para comprender los conceptos del capítulo, se propone realizar una actividad de investigación y simulación.

1. Seleccionar un sistema de control de versiones diferente de Git, como Subversion o Mercurial.
2. Consultar qué herramienta permite interactuar con ese sistema desde Git.
3. Identificar los pasos necesarios para importar un repositorio.
4. Investigar qué información puede conservarse durante la migración, como las confirmaciones, las ramas, las etiquetas y los autores.
5. Describir cómo se verificaría que la migración fue exitosa.
6. Elaborar una conclusión sobre las ventajas y las dificultades de trasladar un proyecto a Git.

Si no se dispone de un repositorio externo para realizar pruebas, la actividad puede desarrollarse de forma documental, explicando el procedimiento sin ejecutar comandos que requieran acceso a otro servidor.

Conclusión

El capítulo 9 de *Pro Git* explica que Git puede utilizarse no solo para administrar proyectos nativos, sino también para interactuar con otros sistemas de control de versiones y para migrar proyectos existentes.

Las herramientas de integración permiten que los equipos continúen trabajando con infraestructuras anteriores mientras aprovechan las capacidades de Git. Por otro lado, los procesos de migración ofrecen una alternativa para adoptar Git como sistema principal, procurando conservar el historial y los datos importantes del proyecto.

En ambos casos, es indispensable revisar la compatibilidad de las herramientas, verificar la información importada y establecer procedimientos claros para compartir cambios. De esta manera, se puede realizar una transición organizada y reducir el riesgo de pérdida de información.

Referencias

Chacon, S. y Straub, B. *Pro Git*, segunda edición, capítulo 9: «Git y otros sistemas».

Git como cliente: https://git-scm.com/book/es/v2/Git-y-Otros-Sistemas-Git-como-Cliente

Migración a Git: https://git-scm.com/book/es/v2/Git-y-Otros-Sistemas-Migración-a-Git